

Week - 2 Module

1 Find Student Roll-No. (Linear Search)

Problem Statement

Your class teacher has a list of student roll number stored in the order they registered. Since the list is unsorted, you need to find whether a particular student roll no. exists.

Write a program to search for the given roll no. using linear search.

```
#include <stdio.h>      #include <stdio.h>
#include <conio.h>      #include <stdlib.h>

int main () {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    int roll_numbers[N];
    for (int i = 0; i < N; i++) {
        if (scanf("%d", &roll_numbers[i]) != 1)
            return 0;
    }

    int x;
    if (scanf("%d", &x) != 1) return 0;
```



```

bool found = false;
for (int i = 0; i < N; i++) {
    if (arr_numbers[i] == x) {
        printf("Found at index %d\n", i);
        found = true;
        break;
    }
}
if (!found) {
    printf("Not Found\n");
}
return 0;
}

```

- Logic:-> uses a simple for loop to check each element sequentially.

For example

Input

5 1 2 3 4 5

Output

Found at index 3

2 Search Product ID (Binary search)

Problem Statement

An online store keep Product IDs sorted in ascending order. A customer wants to know whether a Particular Product exists in inventory.

Write a Program to search the Product using (Binary search).

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int main () {
```

```
    int N;
```

```
    if (scanf ("%d", &N) != 1) return 0;
```

```
    int product_ids[N];
```

```
    for (int i = 0; i < N; i++) {
```

```
        if (scanf ("%d", &product_ids[i]) != 1) return 0;
```

```
    }
```

```
    int target;
```

```
    if (scanf ("%d", &target) != 1) return 0;
```

```
    int low = 0;
```

```
    int high = N - 1;
```



```
bool found = false;
```

```
while (low <= high) {  
    int mid = low + (high - low) / 2;
```

```
    if (Product_ids[mid] == target) {
```

```
        found = true;
```

```
        break;
```

```
    } else if (Product_ids[mid] < target) {
```

```
        low = mid + 1;
```

```
    } else {
```

```
        high = mid - 1;
```

```
    }
```

```
}
```

```
if (found) {
```

```
    printf("Product Available \n");
```

```
} else {
```

```
    printf("Product Not Available \n");
```

```
}
```

```
return 0;
```

```
}
```

*

For example

Input => 10 20 30

Output => Product Available

3 Arrange Exam Score (Bubble sort)

Problem Statement

A teacher wants to arrange student's exam scores in ascending order to prepare the result sheet. Write a program using Bubble sort.

```
#include <stdio.h>
```

```
int main () {
```

```
    int N;
```

```
    if (scanf ("%d", &N) != 1) return 0;
```

```
}
```

```
    for (int i = 0; i < N - 1; i++) {
```

```
        for (int j = 0; j < N - i - 1; j++) {
```

```
            if (scores[j] > scores[j+1]) {
```

```
                int temp = scores[j];
```

```
                scores[j] = scores[j+1];
```

```
                scores[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < N; i++) {
```

```
        printf ("%d", scores[i]);
```

```
        if (i < N - 1) {
```


printf("\n");
 }
 }
 printf("\n");
 return 0;
 }

For example

Input

89 45 78

Output

~~89 45 78~~ 45 78 89

4 Arrange Library Book Numbers (Insertion sort)

Problem Statement

A librarian receives book serial numbers one by one and wants to place each new book in the correct position.

Write a program using Insertion sort to arrange book numbers.


```
#include <stdio.h>
```

```
int main () {
```

```
    int N;
```

```
    if (scanf ("%d", &N) != 1) return 0;
```

```
    int books[N]
```

```
    for (int i=0; i<N; i++) {
```

```
        if (scanf ("%d", &books[i]) != 1) return 0;
```

```
    }
```

```
    for (int i=1; i<N; i++) {
```

```
        int key = books[i];
```

```
        int j = i - 1;
```

```
        while (j >= 0 & books[j] > key) {
```

```
            books[j+1] = books[j];
```

```
            j = j - 1;
```

```
        }
```

```
        books[j+1] = key;
```

```
    }
```

```
    for (int i=0; i<N; i++) {
```

```
        printf ("%d", books[i]);
```

```
        if (i < N - 1) {
```

```
            printf (" ");
```


}

}

printf("\n")

return 0;

For example

55 23 87

~~55 23 87~~ 23 55 87

5 Rank Players by score (selection sort)

Problem statement

a gaming club wants to rank players based on scores from lowest to highest.

Write a program using selection sort.

#include <stdio.h>

int main () {

int N;

if (scanf ("%d", &N) != 1) return 0;

int Player_Scores [N];

for (int i = 0; i < N; i++) {

if (scanf ("%d", &Player_Scores[i]) != 1) return 0;


```

}
for (int i = 0; i < N - 1; i++) {
    int min_idx = i;
    for (int j = i + 1; j < N; j++) {
        if (Player.scores[j] < Player.scores
            [min_idx]) {
            min_idx = j;
        }
    }
    int temp = Player.scores[min_idx];
    Player.scores[min_idx] = Player.scores[i];
    Player.scores[i] = temp;
}
for (int i = 0; i < N; i++) {
    printf("%d ", Player.scores[i]);
    if (i < N - 1) {
        printf(" ");
    }
}
printf("\n");
return 0;
}

```

for example

input → 70 20 90

output → ~~70 20 90~~ 20 70 90


```

1  #include <stdio.h>
2  #include <stdbool.h>
3
4  int main() {
5      int N;
6      // Read the number of students
7      if (scanf("%d", &N) != 1) return 0;
8
9      int roll_numbers[N];
10     // Read the N roll numbers
11     for (int i = 0; i < N; i++) {
12         if (scanf("%d", &roll_numbers[i]) != 1) return 0;
13     }
14
15     int X;
16     // Read the roll number to search
17     if (scanf("%d", &X) != 1) return 0;
18
19     bool found = false;
20     // Perform Linear Search
21     for (int i = 0; i < N; i++) {
22         if (roll_numbers[i] == X) {
23             printf("Found at index %d\n", i);
24             found = true;
25             break; // Stop searching once found
26         }

```



```
27     }
28
29     // If the element was not present in the array
30     if (!found) {
31         printf("Not Found\n");
32     }
33
34     return 0;
35 }
```


Output

Status: **Accepted**

Memory Used: 720 byte Execution Time: 0.003 sec

Found at index 3

Input

5
101 105 113 110 130
110


```

1  #include <stdio.h>
2  #include <stdbool.h>
3
4  int main() {
5      int N;
6      // Read the total number of items
7      if (scanf("%d", &N) != 1) return 0;
8
9      int product_ids[N];
10     // Read the sorted product IDs
11     for (int i = 0; i < N; i++) {
12         if (scanf("%d", &product_ids[i]) != 1) return 0;
13     }
14
15     int target;
16     // Read the target ID to search for
17     if (scanf("%d", &target) != 1) return 0;
18
19     int low = 0;
20     int high = N - 1;
21     bool found = false;
22
23     // Perform Binary Search
24     while (low <= high) {
25         int mid = low + (high - low) / 2;
26

```

```

17     if (product_ids[mid] == target) {
18         found = true;
19         break; // Element found, exit loop
20     }
21     else if (product_ids[mid] < target) {
22         low = mid + 1; // Search in the right half
23     }
24     else {
25         high = mid - 1; // Search in the left half
26     }
27 }
28
29 // Output Format
30 if (found) {
31     printf("Product Available\n");
32 } else {
33     printf("Product Not Available\n");
34 }
35
36 return 0;
37 }

```



```

1  #include <stdio.h>
2
3  int main() {
4      int N;
5      // Read the total number of scores
6      if (scanf("%d", &N) != 1) return 0;
7
8      int scores[N];
9      // Read N space-separated scores
10     for (int i = 0; i < N; i++) {
11         if (scanf("%d", &scores[i]) != 1) return 0;
12     }
13
14     // Bubble Sort Algorithm to sort in ascending order
15     for (int i = 0; i < N - 1; i++) {
16         for (int j = 0; j < N - i - 1; j++) {
17             if (scores[j] > scores[j + 1]) {
18                 // Swap the adjacent elements
19                 int temp = scores[j];
20                 scores[j] = scores[j + 1];
21                 scores[j + 1] = temp;
22             }
23         }
24     }
25 }

```

```
27     for (int i = 0; i < N; i++) {
28         printf("%d", scores[i]);
29         if (i < N - 1) {
30             printf(" "); // Print space between numbers
31         }
32     }
33     printf("\n");
34
35     return 0;
36 }
```


Status: **Accepted**

Memory Used: 736 byte | Execution Time: 0.003 sec

12 45 78 89 99

Input

5
89 45 78 12 99

```

1  #include <stdio.h>
2
3  int main() {
4      int N;
5      // Read the number of books
6      if (scanf("%d", &N) != 1) return 0;
7
8      int books[N];
9      // Read N book serial numbers
10     for (int i = 0; i < N; i++) {
11         if (scanf("%d", &books[i]) != 1) return 0;
12     }
13
14     // Insertion Sort Algorithm
15     for (int i = 1; i < N; i++) {
16         int key = books[i];
17         int j = i - 1;
18
19         /* Move elements of books[0..i-1], that are
20          greater than key, to one position ahead
21          of their current position */
22         while (j >= 0 && books[j] > key) {
23             books[j + 1] = books[j];
24             j = j - 1;
25         }
26         books[j + 1] = key;
27     }

```



```
29 // Output Format: Print sorted book numbers separated by a space
30 for (int i = 0; i < N; i++) {
31     printf("%d", books[i]);
32     if (i < N - 1) {
33         printf(" ");
34     }
35 }
36 printf("\n");
37
38 return 0;
39 }
```

Status: **Accepted**

Memory Used: 596 byte | Execution Time: 0.003 sec

23 34 55 87 111

Input

55 23 87 111 34


```

1  #include <stdio.h>
2
3  int main() {
4      int N;
5      // Read the total number of players
6      if (scanf("%d", &N) != 1) return 0;
7
8      int player_scores[N];
9      // Read N player scores
10     for (int i = 0; i < N; i++) {
11         if (scanf("%d", &player_scores[i]) != 1) return 0;
12     }
13
14     // Selection Sort Algorithm
15     for (int i = 0; i < N - 1; i++) {
16         int min_idx = i;
17
18         // Find the minimum element in the remaining unsorted array
19         for (int j = i + 1; j < N; j++) {
20             if (player_scores[j] < player_scores[min_idx]) {
21                 min_idx = j;
22             }
23         }
24     }

```

```
26     int temp = player_scores[min_idx];
27     player_scores[min_idx] = player_scores[i];
28     player_scores[i] = temp;
29 }
30
31 // Output Format: Print sorted scores separated by a space
32 for (int i = 0; i < N; i++) {
33     printf("%d", player_scores[i]);
34     if (i < N - 1) {
35         printf(" ");
36     }
37 }
38 printf("\n");
39
40 return 0;
41 }
```

Status: **Accepted**

Memory Used: 796 byte | Execution Time: 0.003 sec

10 20 40 70 90

Input

5
70 20 90 10 40